PolyPaint: Architecture & UI Critique III

What landed, what landed differently, and what is genuinely left

Prepared from a full-tree review; every claim re-verified against source

July 2026 — verified against `main` at `542b5b4` (July 3)

Executive summary

This is the third issue of the critique, and the first in which the system's *largest standing risk from issues I–II is structurally retired.* The four-representation program pipeline (text → chips → tokens → C VM, mirrored in a hand-written JS catalog) no longer exists in that form: chips are gone as an authoring surface, every frontend vocabulary is generated from a backend registry, and text source — parsed, compiled, fingerprinted, and adversarially oracle-tested — is the single way a program enters the system. The June–July hardening arc (code-reviews 12–23; 191 commits since issue II) also split the monolith into 13 ordered parts, established that the frontend has *never* contained WASM or Web Workers (correcting stale project records inherited from PolyPaint's in-browser predecessor app), converted all six program editors to one shared text-first layout, and grew the verification stack into something rarer than the last issue knew: *frozen legacy oracles* that pin today's parsers against yesterday's, byte-for-byte, and a gate-completeness meta-test that makes "forgot to wire the test in" a CI failure.

Three findings dominate this issue:

1. **The architecture half of critiques I–II is done — several items landed *differently* than proposed, and mostly better.** Side-by-side text↔chips (old alternative B) was overtaken by text-only editors with generated Starter/Help panels; the chip inspector (old A) shipped as a double-click token inspector over generated help registries; "make text the only source" (old D5-4, flagged as far-future) simply happened. The proposals were directionally right and specifically wrong — a good outcome for a critique.
2. **The workflow half is untouched and is now the whole backlog.** Jobs are still polled text lines; results are still a row table; parameters are still typed, not dragged; there is still no command palette. Every one of these has become *cheaper* to build since issue II, because the pieces they need (generated help registries with lookup maps, a uniform editor component, lean list endpoints, mosaic walls) now exist for other reasons.
3. **The remaining risks are small and specific, not structural.** A three-generator gap in the predeploy gate, dead chip machinery awaiting deletion, one editor missing debounced validation, and the two long-standing verification legs (property-based VM parity; the existing-but-ungated Playwright specs). This review found no new drift class.

1 Method and lineage

Issue I (June 12) reviewed the tree after the first implementation sprint; issue II (June 17) corrected two claims (param text mode did not yet exist; stack-effect metadata was not yet

exported). This issue was prepared by re-verifying *every* factual claim of issue II against `main` at `542b5b4`: two full-tree sweeps (frontend feature-by-feature; backend/deploy fact-by-fact) plus targeted reads of every file cited. Where this document says "confirmed absent" it means a grep/read returned nothing, not that memory says so. Since issue II the tree absorbed, in order: the no-WASM lineage correction (June 19; see D1), the text-only Param/Coeff editors (June 22), the coeff–param registry convergence and help system (late June), the code-review-23 hardening batch — 37 findings, waves A–H, all closed (July 2) — and the Root/solve-score editor unification (July 3).

2 System inventory

Numbers measured on `main` at `542b5b4`.

Component	
Frontend total (index.html 3,207 + 13 js/ parts)	22
JS function declarations	
Generated JS bundles (vocab/catalog/profiles/opcodes)	9 (+ OpenSeadrag
Python modules under lambda/	
Native C sources (33 .c + 37 .h)	70 files, 175
Largest C source (sweep_cli.c: solver + 2 program VMs)	10
Deployed Lambda functions (one manifest)	
API Gateway routes (POST)	
Step Functions state machines / binary layers	
deploy.sh (idempotent, gates inline)	1,993 lines (data in deploy_man
Test files / predeploy gate	143 files; gate runs 72 + JS harness: 860 passed, 7
Frozen legacy oracle modules	5 (parser/compiler snapshots, by
Program DSLs (param, coeff, root, solve-score)	4 — text s

Notable deltas from issue II: routes 48 → 60 (program-source compile routes, mosaic share, deepzoom management); DSLs 2 → 4 (root and solve-score gained full source layers); test files 116 → 143; the frontend grew ∼2.5k lines while *deleting* chips, WASM, and workers — the growth is editors, help generation, and mosaics.

3 Architecture: verified verdicts

Sections D1–D9 carry over from issues I–II; each verdict below is the *re-verified* current state. D10 is new.

3.1 D1 — Buildless vanilla JS, 13 ordered parts

Confirmed as described in issue II, with the split now carrying real weight: 13 classic-script parts (943–2,655 lines) registered in `window.__ppParts` with a load-order consistency check, plus 9 generated bundles, all under build-versioned asset keys (`assets/<BUILD_ID>/...`) so a deploy flips atomically. A lineage correction this issue owes the record: **PolyPaint's frontend never contained WASM or Web Workers** — zero references even in the pre-split 19,989-line monolith. The in-browser solver (`step_loop.c` → WASM, multi-worker fast mode, 40 WASM tests) belongs to PolyPaint's *predecessor*, the standalone root visualizer that still lives beside it in the repository; stale project notes carried its WASM lore forward until code-review-18 (June 19) verified none of it applies here. The browser is, and always was, a thin client: programs compile in Python, execute in C; the page edits text, posts JSON, and polls.

Verdict: Working as intended. The ESM conversion (stage 2) remains unstarted and is now *less* urgent: the parts have stabilized, the harness executes them in-context, and the state surface shrank when chips died. Still missing — and still cheap — is static analysis: **no ESLint, no tsc –checkJs, no lint config exists anywhere in the tree** (verified). That gate would have caught the serializer recursion of issue I and costs days.

3.2 D2 — Manifest-driven bash deployer

Confirmed and stable. `deploy_manifest.json` (711 lines) carries 41 function specs and 60 routes; `deploy_manifest.py` validates uniqueness, single-owner route ownership, layer whitelists, retired-name collisions, and the layer-only `LD_LIBRARY_PATH` rule (the D9 incident lesson, now a schema rule). deploy.sh is 1,993 lines and emits its own sources from the manifest. `show-build` and the post-deploy INIT sweep close the loop after every deploy.

Verdict: Done; unchanged since issue II except growth (60 routes). The one refinement worth taking: three generators still run only inside deploy.sh (tri/long palette catalogs, coeff function catalog) rather than under the predeploy `--check` pattern the other six follow — see N5.

3.3 D3 — 41 Lambdas, shared code copied per zip

Confirmed unchanged: exactly three *binary* layers exist (libvips, LAPACK, pdf-python); there is still **no shared-code layer**, and `shared.py`, `solve_score_chain.py`, palette-name modules, and sidecar helpers are copied into zips per function (finalize alone bundles seven helper modules). The mid-deploy version-skew window is real but has never bitten; the packaging tests and the lean-bundle discipline (a helper added to a shared module can break ten zips, which is why `solve_score_chain` deliberately keeps a local number-format copy, parity-pinned) contain it.

Verdict: Keep the topology. The shared-code layer remains the cheapest open architecture item, but its payoff shrank as the packaging tests matured; it is now hygiene, not risk reduction. Do it opportunistically, not urgently.

3.4 D4 — Python around native C via JSON-over-stdin

Confirmed, and the "generate the mirrors" program is complete well beyond what issue II recorded. **Eight generators** now emit every cross-language mirror: opcodes (`merged_opcodes` → Python + C header + JS), program profiles (→ Python + JS), four registry vocabularies (param, coeff, solve-score, root → JS), the coefficient function catalog (→ C header + JS), and parity fixtures. The last hand-written frontend catalog (`_rtCatalog` for root transforms) was deleted on July 3; the root registry gained ui blocks whose descriptions were written from the C implementations in `root_xforms.h`, not from memory.

Verdict: Done. The remaining hand-synchronized C surface (fn_index packing ladders, metric tables, caps) is drift-pinned test-by-test — partition tables, min-roots table, root cap pinned three ways (profile = Python = parsed C `#define`). The correct next step is not more generation but property-based parity (N7): random programs through Python fold vs C eval.

3.5 D5 — The program DSLs: two became four, and text won

The biggest divergence between proposal and outcome. Issue II's alternative 4 — "make text the only source and render chips as a read-only visualization; viable only after the side-by-side editor exists" — was implemented *directly*, skipping the intermediate: Param and Coeff chips were dropped June 22, Root chips July 3; solve-score chips survive only as read-only strips inside the

saved program modal. Every kind gained a real source layer: parser with line/column diagnostics and stable machine codes, compiler, fingerprint, and an HTTP compile route (`/compile-{param, coeff, root, solve-score}-program-source`). Root and solve-score joined param/coeff as first-class program kinds under a shared `program_profiles` registry.

Two properties of the shipped design deserve the record, because they were not in any proposal:

- **Spelling preservation.** Loading a saved program and saving it back is byte- and fingerprint-identical (the original row is stashed at hydration and re-emitted verbatim while pristine); any edit respells. This closed a whole family of "load then save churns the artifact key" defects.
- **Wire-preserving validation.** Compile-time additions (e.g. the static division-by-zero guard) are validators, never folders: emitted tokens stay byte-identical for every previously valid program. The cache keys make this discipline mandatory; the review cycle made it explicit policy.

Verdict: Settled, and better than proposed. Four kinds, one editor pattern, all vocabularies generated, chips retired. The two C program VMs remain two (plus the root/solve-score evaluators) — unification is still correctly queued behind a fingerprint version bump that no feature yet demands. Residue: the dead chip-editing machinery (add/move/remove, editable-chip renderers, orphaned CSS) is still in the tree — see N2.

3.6 D6 — Fingerprint cache and format generations

Correction to issues I–II, which claimed "nothing in the key encodes format generation." Verified: both param and coeff fingerprints hash `{"version": 1, "execution_spec": ...}` — a version field exists *inside* the hashed payload, so a deliberate global invalidation is a one-line bump. What does not exist is per-artifact dual-read migration. Separately, program *objects* now carry `spec_version: 2` and `"v2;"`-prefixed spec strings (v1→v2 migration routes shipped and lifecycle-tested), so the program-identity layer already did the migration this section once asked for.

Verdict: Keep. The freeze is real but the escape hatches exist at both layers. Write down the bump procedure (when to bump, what re-renders) rather than building more mechanism.

3.7 D7 — SFN + DynamoDB status + polling

Confirmed unchanged: the browser's only live signal is still HTTP polling (3-second `setInterval` on `/check-status`); there is no WebSocket, SSE, or push anywhere; no route surfaces Step Functions execution history (the only SFN API called anywhere is `start_execution`). Two localStorage records let an active render/palette run survive a reload — resume, not visibility.

Verdict: Unchanged, and now the clearest architecture–UI gap in the system: the backend knows job topology precisely and the frontend shows appended text lines. The jobs rail (old C, unbuilt) plus a cheap read-only execution-history route remains the recommendation, in that order.

3.8 D8 — The verification stack

Substantially stronger than issue II recorded, and one of its two "missing legs" turns out to exist:

- **Gate completeness is now enforced, not hoped for.** `test_predeploy_gate_completeness.py` pins: every `tests/test_*.py` on disk must be in the predeploy gate or in a frozen allow-list — a new test file that isn't wired in fails CI. Issue II's H8 observation ("100 of 140 files ungated")

is not just fixed; the failure mode is retired. The gate now runs 72 test files + the JS harness: 860 passed, 77 subtests at HEAD.

- **Frozen oracles.** Five `*_legacy.py` modules (4,494 lines) snapshot the param/coeff parsers and compilers; current parsers are tested against them on a harvested corpus with pinned counts (so "0 cases checked" can never pass), under an explicit never-edit policy. This is the strongest anti-regression device in the tree.
- **Playwright exists.** Seven E2E specs (`tests/e2e/`: compute, results, palette, render, render-solve-score, favorites, deepzoom-inventory) with a config and pinned `@playwright/test` — present since April, invoked manually (`npx playwright test`), *not* wired into any gate. Issue II called this leg missing; it is actually built and unplugged (N6).
- **Property-based testing** remains genuinely absent (no hypothesis anywhere) — the one leg still missing.

Verdict: The standout, now with teeth (meta-gated, oracle-pinned). Remaining: wire the existing Playwright specs into a gate; add property-based VM parity.

3.9 D9 — Native library exposure

Confirmed settled and untouched since issue II: four link modes, the manifest-level `LD_LIBRARY_PATH` rule, Docker-gate loader assertions over 14 deploy binaries, layer hygiene pins, and the INIT sweep (which passed after both July deploys). No new incident.

Verdict: Settled; nothing new to add, which is the point.

3.10 D10 — The adversarial-review method itself (new)

Between issues II and III the project ran five review cycles culminating in code-review-23: 37 verified findings, fixed in waves, each wave gated, committed, and recorded in the review document with commit hashes; findings closed *by decision* are recorded as accepted debt so they cannot silently reopen. Two structural inventions came out of it worth naming as architecture:

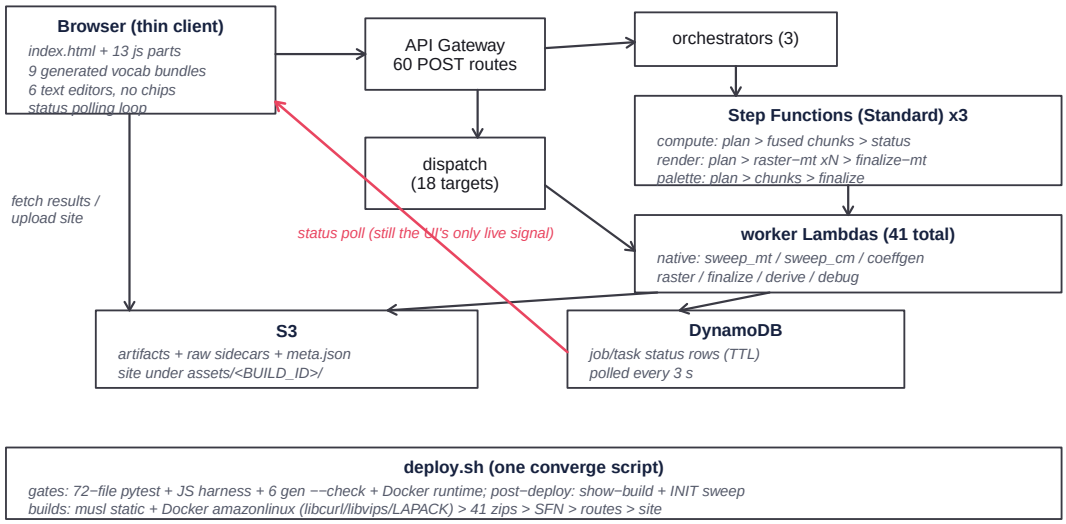
- **Auto-classified diagnostics.** Error types upgrade a bare default code from the message at construction, so ∼90 legacy raise sites gained stable machine codes without editing any site — and the oracle comparison permits refining the legacy default while pinning specific codes strictly.
- **Schema exemptions as tests.** The one intentional runtime/display arity mismatch (`pow`) lived as an explicit allow-listed test exemption until the registry was normalized — after which *deleting the exemption* was the proof of completion. Exceptions are pinned artifacts, not lore.

Verdict: Keep running the method. Its cost is real (the July 2 batch was ∼20 commits of fixes plus documentation) and its yield — every class of drift found before production — has been consistently worth it.

3.11 Decision matrix

Decision	Call	Risk if ignored
D1 buildless, 13 parts	Keep; lint gate still open	unexecuted-path bugs
D2 manifest deployer	DONE + stable	(retired)
D3 41 small Lambdas	Keep; layer is hygiene now	mid-deploy skew (contained)
D4 C core, generated mirrors	DONE (8 generators)	(retired)
D5 four DSLs, text-first	DONE beyond proposal	dead chip code lingers
D6 fingerprint cache	Keep; bump procedure only	undocumented bump
D7 SFN + polling	Unchanged: the open gap	polling UX ceiling
D8 verification stack	Meta-gated + oracles	property leg missing
D9 native lib exposure	Settled	(retired)
D10 adversarial reviews	Keep the cadence	drift returns quietly

4 The workflow as built

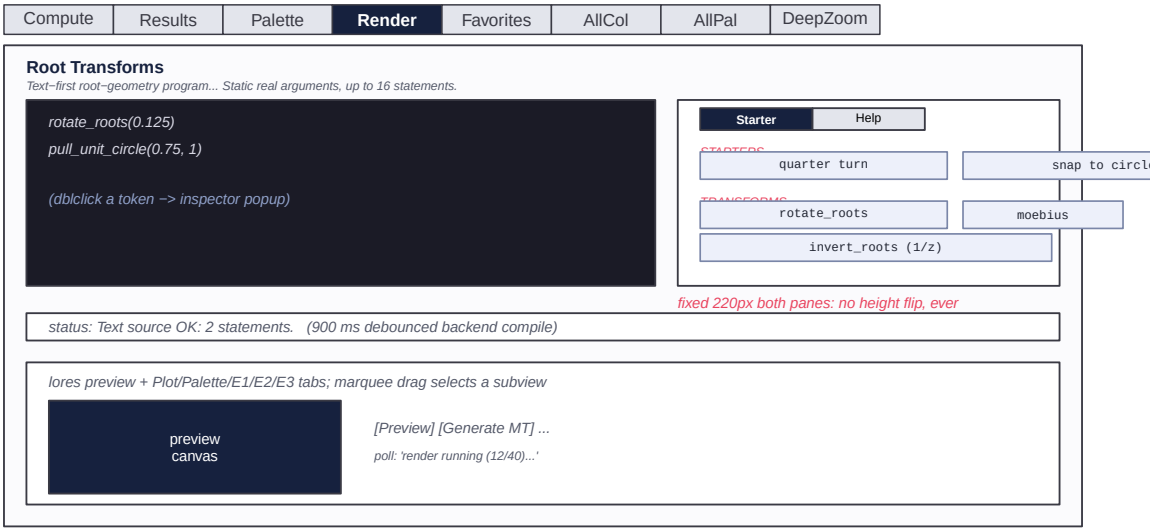


The loop's pain point is unchanged and now singular: everything between "dispatched" and "done" is a 3-second poll rendered as text. The compute side of the loop, by contrast, tightened considerably — edit text, get a 900 ms-debounced compile verdict inline, preview without leaving the tab.

5 The UI as built (rewritten)

The description in issues I–II is obsolete. There are eight tabs (Compute, Results, Palette, Render, Favorites, AllCol, AllPal, DeepZoom). Programs are edited in **six identical text editors** — Param and Coeff on Compute; solve-score and Root on Render, each with a Palette-tab twin — sharing one layout system: a fixed 220 px monospace textarea beside a fixed 220 px side panel with Starter/Help tabs. Starter holds generated snippet buttons (cursor-aware insertion); Help renders a generated reference (statement forms, every metric/transform/op with parameters, allowed sources, ranges, examples) from the same vocabularies the compilers use. Double-clicking any token opens an inspector popup for that symbol. Four of the six editors validate on a

900 ms debounce against the real backend compiler and report `Line N: message` in a status line; geometry never shifts — the height-flip complaint of issue II is gone because the mode toggle is gone.



5.1 What was fixed since issue II, in the constraint list's own terms

Of the eight "why it feels constraining" items from issues I–II:

3. *Editing in 12-character boxes* — **gone**. The inline chip inputs no longer exist; editing is a real textarea with a documented language.
4. *Binary Chips/Text toggle* — **gone**, resolved in text's favor everywhere, including Param (which issue II noted had no text mode at all: it now has parser, endpoint, editor, help).
5. *Catalog browsing by native* `<select>` — **gone**. The add-selects were deleted with the chips; the catalog surface is now the Starter buttons and the generated Help reference. What did *not* arrive is fuzzy search over the catalog (see A/N9).
2. *Stack machine invisible* — **half-fixed**: the compiler reports statement counts, token counts, and errors with line/column attribution (solve-score chain errors now land on their statement, not line 1); but there is still no per-statement depth gutter (N8).

Items 1 (layout mirrors wire, not workflow), 6 (jobs are log lines), 7 (no direct manipulation), and 8 (fixed geometry) are **unchanged** — they are the workflow half, and nothing in the hardening arc touched them.

6 UI alternatives: status and revisions

6.1 A — Command palette + inspector: half landed, differently

The inspector half shipped as the double-click token inspector plus the Help tabs — and, importantly, over *generated* registries with a lookup map per program kind, which is exactly the index a command palette needs. The `Ctrl-K` palette itself is confirmed absent (no global key handler exists). The revised proposal is therefore smaller than the original: a fuzzy overlay over (a) the four help-registry lookup maps — already built, already tested for coverage — and (b) a

hand-registered action list (run compute, switch tab, open saved programs). The hard part of A was the metadata; it now exists.

6.2 B — Side-by-side text+chips: superseded (text won)

Do not build. Text-only editors with generated Starter/Help shipped instead and resolved the same complaints with less surface. Two residues of B remain worth keeping as separate small items: the per-statement stack-depth gutter (N8) and inline error highlighting (the status line says `Line 3:` but line 3 is not marked in the textarea).

6.3 C — Jobs rail: open, unchanged, still first

Verified: no persistent cross-tab job surface exists; status is per-tab text + two localStorage resume records; polling is per-run `setInterval`. Everything from issues I–II stands, including the build order (poll-driven rail first, push later). This remains the highest-value unbuilt feature in the system.

6.4 D — XY scrub pad: open, unchanged

Verified absent. The lores-preview Lambda is deployed and fast; the only direct manipulation today is the marquee drag-to-subview on preview images (which proves the event plumbing works). The pad remains the highest delight-per-effort item.

6.5 E — Gallery + compare: partially arrived from an unexpected direction

Results proper is still a sortable row table with a single-thumbnail sidebar and lean-list/detail loading. But the *mosaic* surfaces — AllCol/AllPal tabs as OpenSeadragon walls of every color/palette artifact, plus a standalone shareable `artifact_mosaic_viewer` — deliver the "see everything at once" half of E. What is still missing is the judgment half: a compare tray (2–4 artifacts, matched zoom, synced pan) and faceted filtering. The revised E is: keep the row table as the working list, add compare on top of the mosaic/detail surfaces that now exist.

6.6 F — Studio shell: open; prerequisites improved

Unchanged proposal; note only that its panels are more real than in June (uniform editor component, help registries, mosaic viewers), and that panel geometry is still fully hardcoded (no splitters, no persistence — verified).

7 New findings and ideas (this review)

Small, specific, ordered by value; N1–N6 are cleanups/gaps found by the verification sweeps, N7–N11 are forward ideas.

N1. Unify the editor plumbing. Six editors share one DOM pattern but four hand-copied JS stacks: per-kind textarea getters, status setters, debounce timers, auto-synth flags, snippet inserters (∼4× the same 60 lines). A single `makeProgramEditor(profile)` factory would collapse them and make the seventh editor (if ever) a registration, not a copy. The July 3 work created the fourth copy knowingly; the pattern is now proven and ready to fold.

N2. Delete the dead chip machinery. With chips gone as an authoring surface, `addChip`/`removeChip`/`moveChip`/`updateChipParam`, the editable-chip renderer paths, the never-mounting picker popups, and orphaned CSS are dead code in the hottest file. Delete and pin absent (the harness already knows the pattern).

N3. Solve-score debounced validation. The one asymmetry left among the six editors: solve-score still validates only via its Compile Text button while the other four kinds validate as you type. Same 30-line pattern; the endpoint exists.

N4. Saved root programs — or document the asymmetry. Param, coeff, and solve-score have save/list/fetch/delete routes and modals; root has none (root programs persist only inline in artifact metadata). Either add the fourth CRUD (cheap: the storage handler pattern is uniform) or record "root programs are per-artifact by design" so nobody reads it as an accident.

N5. Gate the last three generators. tri/long palette catalogs and the coeff function catalog regenerate inside deploy.sh but have no `--check` in predeploy, unlike the other six generators — a hand-edit to those bundles would surface at deploy time, not review time.

N6. Plug in Playwright. Seven specs exist and run manually; they are in no gate. Wire them as a deploy-time gate (they need a served site, so predeploy is the wrong slot) or at minimum a checklist step with the exact command — issue II thought this leg was missing; it is merely unplugged.

N7. Property-based VM parity. Generate random well-typed programs per kind; assert Python static-fold vs C execution agreement within tolerance, and parser round-trip (source→chain→source) idempotence. The G9/H4/H6 bug family — found by hand, one instance at a time — is exactly what this automates. Hypothesis is one dependency; the harness patterns exist.

N8. Per-statement depth gutter + inline error marks. The compile endpoints already return statement-attributed diagnostics; returning per-statement stack depth too would let the editors paint a gutter and highlight the failing line — the last useful residue of old alternative B, at a fraction of its cost.

N9. Ctrl-K over the help registries. The palette's index problem solved itself: four generated lookup maps with coverage tests. An overlay + score function + action list is days of work and finally makes the whole surface searchable.

N10. Jobs rail, then push. Unchanged from issues I–II and still first among the large items: the data exists, the UI is text. Build it on polling; upgrade transport later.

N11. Compare tray on the mosaic/detail surfaces. The E revision: 2–4 pinned artifacts at matched zoom with synced pan, fed by the existing detail routes and OpenSeadragon instances. The mosaics answer "what exists"; compare answers "which is better" — the last unbuilt piece of the judgment loop.

8 Roadmap

Status	Item	Payoff
DONE	manifest deploy + validator + INIT sweep + show-build	deploy refuses bad data; outages loca
DONE	generated mirrors: 8 generators, all vocabularies	drift classes retired at the source
DONE	text-first editors x6 + Starter/Help + dblclick inspector	one editing idiom; no mode flips; doc
DONE	source layer for all 4 kinds + coded diagnostics	programs are text: diffable, saveable
DONE	oracle freeze + gate-completeness meta-test	regressions cannot pass silently or do
open	N2/N1/N3/N5: dead chips, editor factory, ss debounce, gen gate (low)	less dead surface; one editor impleme
open	N6/N7: Playwright gate + property-based parity (low/med)	the two missing verification legs
open	N9: Ctrl-K palette over help registries (low)	whole surface searchable
open	N10: jobs rail — poll first, push later (med)	work in flight becomes visible
open	D + N8: XY scrub pad; depth gutter + inline errors (med)	exploration becomes tactile; errors la
open	N11 + F: compare tray; studio shell last (med/high)	judgment loop closes; layout matches

9 Closing

Issue I asked the architecture to stop hand-copying facts; issue III can report that it did — eight generators, five frozen oracles, a meta-gated test suite, and a program pipeline whose four representations collapsed to one authored form with generated mirrors. It asked the UI for a real editing surface; the tree answered with something simpler and stronger than the proposal: six identical text editors with generated documentation, validation as you type, and no chips at all. The critique process itself became part of the architecture (D10), and its most recent full cycle closed all 37 of its own findings.

What remains is precisely what none of the hardening touched: the workflow layer. Jobs are still text lines; comparison is still squinting; parameters are still typed. The good news is structural — every remaining item is cheaper than when first proposed, because the machinery built for correctness (registries, lookup maps, lean routes, mosaic viewers) is exactly the machinery the workflow features need. The next unit of work that changes how the tool *feels* is the jobs rail; the next that changes how it *fails* is a lint gate plus property-based parity; and the cheapest way to make the whole surface feel modern in a day remains `Ctrl-K` over the help registries that already know every symbol in the system.